

INDIVIDUAL CONTRIBUTION REPORT

AudioLIT - Multimodal Explainable-AI Workbench for Speech Recognition, Emotion Analytics and Deepfake Detection

Name: Perera D.I.R.T.

Student ID: 230475N

Group: Group 19

Section	Details
Documentation	• Authored the Software Requirements Specification: functional requirements, non-functional performance targets, and the boundary separating inherited baseline behaviour from new work. • Wrote the technical feasibility study, resource matrix, and the initial proposal and scope definition. • Wrote the README and local execution guide.
Project Management	• Established the repository structure, branch protection, and the continuous-integration pipeline for backend and frontend. • Set up issue-tracker automation and produced the dependency-ordered plan used to sequence the build.
Architecture & Migration	• Restructured the inherited codebase into the layered package design, decoupling routes from domain logic. • Standardised all audio input / output on a single dependency, removing a redundant library. • Prototyped fan-out / fan-in task orchestration, then merged two duplicated orchestrators into one queue fabric.
Model Registry	• Built the dynamic model registry:- identifier resolution, mandatory safetensors enforcement before deserialisation, revision pinning, a bounded cache, and a circuit breaker for hub failures. • Implemented forward and attention hook registration for the supported model families.
Emotion & Deepfake Models	• Integrated the speech emotion recognition model, and evaluated and selected its default checkpoint, resolving an open decision in the requirements. • Implemented the binary deepfake probability head and the forensic feature-map route.
Explainability & Faithfulness	• Implemented the attribution faithfulness routines, high-saliency masking engine, degradation scoring pipeline, and the deletion / insertion AUC metric. • Replaced the inherited silent synthetic-attention fallback with a shared provenance contract, so a synthesised stand-in is tagged rather than presented as model output.
Caching, Performance & Testing	• Built the dataset warm-up engine with time estimation, cancellation and per-item memory cleanup. • Defined every cache key family and payload shape in one place, and added content-addressed hashing so cache identity follows the audio rather than the file path. • Wrote the backend boundary test suite, remediated dataset path-traversal gaps, and maintained pipeline health.

Challenges & Solutions

- Warmed dataset results were unreadable by the interface. The warm-up wrote one payload shape into two cache families expecting different shapes, so a cache hit failed the request instead of recomputing - I separated the contracts and defined each key family once.
- Custom models returned the default model's output - Loaders hardcoded a checkpoint while caching under the requested model's name. I bound the selected model through every loader and made cache keys model-specific.
- Fabricated data was presented as genuine model output - Affected attention, saliency and the latent projection. I added the provenance contract, removed random-data fallbacks, and stopped the auditor scoring anything unmeasured.
- Two independently-passing changes broke the build together - I added a rebase-and-re-verify step and a mechanical check that fails when a change deletes something still referenced.

INDIVIDUAL CONTRIBUTION REPORT

AudioLIT - Multimodal Explainable-AI Workbench for Speech Recognition, Emotion Analytics and Deepfake Detection

Name: Pathirana K.P.R.J.

Student ID: 230467R

Group: Group 19

Section	Details
Documentation	• Drafted the functional requirements for the multi-task explainability overlays and canvas-driven audio mutation. • Authored in Feasibility report, created the diagrams in the design document and created the project gantt chart. • Produced the dataset-management compliance (created data dictionary) audit and kept the issue plan current after each round of merges. • Authored in Test report, created test approach, deliverables and risk,dependencies sections.
Dataset Ingestion	• Dataset Subsampling and Metadata Preparation. • Built the multi-task ingestion core: one streaming loader interface across the speech, emotion and deepfake corpora. • Wrote the loaders completing the seven-corpus registry, adding integrity validation, a bounded working footprint, and licence surfacing for non-commercial corpora. • Added ground-truth upload with order-independent filename matching, and wired the real registry into the live routes.
Acoustic Profiling	• Implemented the probabilistic fundamental-frequency tracker, marking unvoiced frames rather than interpolating through them. • Implemented RMS energy estimation and the amplitude contour, assembling both into the signal-processing profile pipeline.
Canvas & Visualisation	• Developed the canvas waveform visualisation suite and the downsampled buffer rendering mathematics. • Built the 2D spectrogram grid selector and the resolver converting pixel selections into time and frequency units, so the backend never receives pixels. • Implemented the drag and bounding-box tracker and the alpha-blended heatmap overlay layer.
Backend Architecture	• Construct Librosa Digital Signal Processing (DSP) Extraction Pipeline. • Build an automated bias diagnostic tool using non-native English speaker data vectors from the L2-ARCTIC corpus to isolate language mapping errors.
Frontend Architecture	• Led the interface redesign: navigation, sidebar, workspace grid, explainability panels, status bar and dark mode. • Built the task selector for the three model families and the custom model interface. • Consolidated the inference pipeline onto the asynchronous multi-task job, and added per-row regeneration with cache bypass.
Testing & CI	• Wrote the automated backend suites, interface component boundary tests, and wired up the frontend test runner. • Execute Full Software Testing and Multi-Task Data Science Part Evaluation. • Speed up the pipeline by caching dependency downloads and using the processor-only framework build.

Challenges & Solutions

- Different datasets had different structures, annotation formats, and speaker distributions - I inspected each dataset before sampling, normalized annotation-tier detection and performed metadata/audio verification before finalizing each subset.
- The tests were sometimes getting stuck because parallel workers finished in different orders - I fixed this by making the tests independent of worker execution order.
- The builds were slow because they downloaded very large GPU packages even on CPU machines - I fixed this by caching dependencies and using a lightweight CPU-compatible version.
- Audio selections on the screen needed to remain accurate when zoom or sample rate changed - I created accurate conversions in both directions between pixels and audio frequency / time.

INDIVIDUAL CONTRIBUTION REPORT

AudioLIT - Multimodal Explainable-AI Workbench for Speech Recognition, Emotion Analytics and Deepfake Detection

Name: Rahim M.I.

Student ID: 230506M

Group: Group 19

Section	Details
Documentation	- Defined the core system baseline architecture and the infrastructure sizing matrix. - Modelled the global component topology, the asynchronous system architecture, and the task lifecycles and worker broker mappings. - Produced the network-latency profiling, deployment-constraint analysis, and the tensor storage and computational sizing study.
Backend Foundation	- Scaffolded the asynchronous API server, application initialisation and cross-origin middleware. - Built the multi-part file upload streaming route and registered the inference and task-status route groups.
Async Task Infrastructure	- Deployed the task broker and parallel computing infrastructure:- worker scaffolding, the broker hook, and per-family queues. - Implemented the concurrent orchestrator running speech recognition, emotion recognition and deepfake detection together on one clip.
Caching System	- Designed and built the deterministic cache-by-hash architecture. - Implemented the payload hashing middleware that computes a content hash before the request reaches the handler. - Built the tensor serialisation layer with binary encoding and compression, so attribution matrices and attention tensors could be stored efficiently.
Real-Time Task State	- Implemented the streaming and long-polling handlers for task states. - Built the task-status hook, global progress component and live prediction display, so long-running jobs report progress instead of blocking the interface.
Explainability & Perturbation	- Built the attribution library integration underpinning the saliency stack, the Integrated Gradients temporal formulation, and the spatial Grad-CAM registration hooks. - Wrote the LIME / SHAP translators for spectrogram frameworks and the value matrix coordinate transformer. - Coded the signal mutation engine, the time-frequency slice masking and pitch exporter, and the patch segmentation and perturbation matrix generator.
Model Ingestion & Testing	- Implemented the version-pinned model cache. - Wrote unit tests for the cache layer, payload hashing, the queue service, the saliency service and the perturbation service. - Implemented the memory profiling and stress-test suites, and a standalone profiling script.

Challenges & Solutions

- Tests could not depend on a live cache server - This made the caching layer untestable in the pipeline. Introduced an in-memory fake so cache behaviour could be asserted deterministically.
- Large attribution tensors were expensive to cache - I adopted binary encoding with compression above a size threshold, leaving small payloads uncompressed.
- The pipeline installed only a subset of the required packages - Backend jobs failed on imports that worked locally. I corrected the dependency installation step.
- Long-running inference blocked the interface with no feedback - Combined the task broker with progress events and a polling fallback, so the browser follows a job rather than waiting on a request.